

MapStruct em Java/Spring

Guia pratico para entender a estrutura de um mapper, onde usar, como ignorar campos e como aplicar em um CRUD real.

Ideia central: o mapper e a camada de traducao entre o que entra/sai pela API e o que fica persistido no banco.

1. O que e um Mapper?

No backend Java, a Entity representa o modelo interno do banco de dados. O DTO representa os dados que entram ou saem pela API. O mapper converte um objeto no outro sem espalhar varios setters pelo service.

Analogia: a Entity e a ficha interna completa. O DTO e o resumo que voce entrega para outro setor ou para a API. O Mapper e quem traduz a ficha para o resumo correto.

2. Estrutura de pastas mais comum

```
src/main/java/com/carrara/api
|-- controller
|-- service
|-- repository
|-- entity
|-- dto
|   |-- request
|   |-- response
|-- mapper
|-- exception
-- config
```

Para projetos pequenos e medios, deixar os mappers em um pacote **mapper** e um padrao simples e bem aceito. Em sistemas maiores, tambem e comum deixar o mapper dentro do pacote do proprio dominio, por exemplo produto/ProdutoMapper.java.

3. Exemplo de Entity

```
@Entity
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private String descricao;
    private BigDecimal preco;
    private Boolean ativo;
    private LocalDateTime dataCriacao;
    private LocalDateTime dataAtualizacao;

    // getters e setters
}
```

4. DTO de entrada e DTO de saida

Em APIs profissionais, e comum separar DTO de entrada e DTO de saida. O request DTO recebe os dados enviados pelo usuario. O response DTO define o que a API devolve.

```
public class ProdutoRequestDTO {

    @NotBlank
    private String nome;

    private String descricao;

    @NotNull
    private BigDecimal preco;

    // getters e setters
}
```

```
public class ProdutoResponseDTO {

    private Long id;
    private String nome;
    private String descricao;
    private BigDecimal preco;
    private Boolean ativo;
    private LocalDateTime dataCriacao;

    // getters e setters
}
```

5. Estrutura completa de um Mapper

```
@Mapper(componentModel = "spring")
public interface ProdutoMapper {

    ProdutoResponseDTO toResponseDTO(Produto produto);

    List<ProdutoResponseDTO> toResponseDTOList(List<Produto> produtos);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "ativo", ignore = true)
    @Mapping(target = "dataCriacao", ignore = true)
    @Mapping(target = "dataAtualizacao", ignore = true)
    Produto toEntity(ProdutoRequestDTO dto);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "ativo", ignore = true)
    @Mapping(target = "dataCriacao", ignore = true)
    @Mapping(target = "dataAtualizacao", ignore = true)
    void updateEntityFromDTO(
        ProdutoRequestDTO dto,
        @MappingTarget Produto produto
    );
}
```

6. O que faz componentModel = spring?

A anotacao **@Mapper(componentModel = "spring")** faz o MapStruct gerar uma implementacao da interface e registrar essa implementacao como Bean do Spring. Assim, voce injeta o mapper no service pelo construtor.

```
@Service
public class ProdutoService {

    private final ProdutoRepository produtoRepository;
    private final ProdutoMapper produtoMapper;

    public ProdutoService(
        ProdutoRepository produtoRepository,
        ProdutoMapper produtoMapper
    ) {
        this.produtoRepository = produtoRepository;
        this.produtoMapper = produtoMapper;
    }
}
```

7. Entity para ResponseDTO

Este metodo converte o objeto interno do banco para o objeto que sera devolvido pela API.

```
ProdutoResponseDTO toResponseDTO(Produto produto);

public ProdutoResponseDTO buscarPorId(Long id) {
    Produto produto = produtoRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Produto nao encontrado"));

    return produtoMapper.toResponseDTO(produto);
}
```

8. DTO para Entity

Este metodo converte os dados recebidos no request para uma entidade que pode ser salva no banco.

```
Produto toEntity(ProdutoRequestDTO dto);
```

```

public ProdutoResponseDTO cadastrar(ProdutoRequestDTO dto) {
    Produto produto = produtoMapper.toEntity(dto);

    produto.setAtivo(true);
    produto.setDataCriacao(LocalDateTime.now());

    Produto produtoSalvo = produtoRepository.save(produto);

    return produtoMapper.toResponseDTO(produtoSalvo);
}

```

Regra pratica: o mapper copia dados. O service aplica regra de negocio, como ativar produto, definir data de criacao ou validar existencia.

9. Como ignorar campos

Para ignorar um campo, use **@Mapping(target = "campo", ignore = true)**. Isso impede que o MapStruct tente preencher aquele atributo no destino.

```

@Mapping(target = "id", ignore = true)
@Mapping(target = "ativo", ignore = true)
@Mapping(target = "dataCriacao", ignore = true)
@Mapping(target = "dataAtualizacao", ignore = true)
Produto toEntity(ProdutoRequestDTO dto);

```

Esses campos sao ignorados porque normalmente pertencem ao controle interno do sistema: o id e gerado pelo banco, as datas sao geradas pela aplicacao e o status ativo pode ser regra do service.

Campo	Por que ignorar?
id	Normalmente e gerado pelo banco.
ativo	Pode ser regra do sistema, nao do usuario.
dataCriacao	Deve ser definida pela aplicacao.
dataAtualizacao	Deve mudar apenas em atualizacoes.

10. Campo com nome diferente

Quando os nomes sao iguais, o MapStruct mapeia sozinho. Quando sao diferentes, voce informa a origem e o destino.

```

@Mapper(componentModel = "spring")
public interface UsuarioMapper {

    @Mapping(source = "nome", target = "nomeCompleto")
    UsuarioResponseDTO toResponseDTO(Usuario usuario);
}

```

Leia assim: pegue **usuario.nome** e coloque em **dto.nomeCompleto**.

11. Convertendo listas

```
List<ProdutoResponseDTO> toResponseDTOList(List<Produto> produtos);
```

Se o mapper sabe converter um Produto em ProdutoResponseDTO, ele tambem consegue converter uma lista. Internamente, ele gera um loop e chama o metodo de conversao item por item.

12. Atualizando entidade existente com @MappingTarget

No update, voce nao quer criar uma entidade nova. Voce busca a entidade existente no banco e atualiza os campos permitidos. O **@MappingTarget** indica o objeto que sera alterado.

```

@Mapping(target = "id", ignore = true)
@Mapping(target = "ativo", ignore = true)
@Mapping(target = "dataCriacao", ignore = true)
@Mapping(target = "dataAtualizacao", ignore = true)
void updateEntityFromDTO(
    ProdutoRequestDTO dto,
    @MappingTarget Produto produto
);

public ProdutoResponseDTO atualizar(Long id, ProdutoRequestDTO dto) {
    Produto produto = produtoRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Produto nao encontrado"));

    produtoMapper.updateEntityFromDTO(dto, produto);
    produto.setDataAtualizacao(LocalDateTime.now());

    Produto produtoAtualizado = produtoRepository.save(produto);
    return produtoMapper.toResponseDTO(produtoAtualizado);
}

```

13. Ignorar campos nulos no PATCH

Em atualizacao parcial, se o usuario envia apenas o preco, voce provavelmente nao quer que nome e descricao virem null. Para isso, use NullValuePropertyMappingStrategy.IGNORE.

```

@BeanMapping(
    nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IGNORE
)
@Mapping(target = "id", ignore = true)
@Mapping(target = "ativo", ignore = true)
@Mapping(target = "dataCriacao", ignore = true)
@Mapping(target = "dataAtualizacao", ignore = true)
void updateEntityFromDTO(
    ProdutoRequestDTO dto,
    @MappingTarget Produto produto
);

```

14. Mapper usando outro Mapper

Quando um objeto possui relacionamento com outro, por exemplo Pedido possui Cliente, voce pode fazer um mapper usar outro mapper.

```

@Mapper(
    componentModel = "spring",
    uses = {ClienteMapper.class}
)
public interface PedidoMapper {

    PedidoResponseDTO toResponseDTO(Pedido pedido);
}

```

15. O que evitar dentro do Mapper

Nao coloque regra pesada no mapper. Ele deve traduzir objetos. Validacoes de negocio, buscas no banco, envio de e-mail, chamada de API externa e calculos complexos pertencem ao service ou a classes especificas.

16. Modelo base para copiar

```

@Mapper(componentModel = "spring")
public interface NomeMapper {

    NomeResponseDTO toResponseDTO(Nome entity);

    List<NomeResponseDTO> toResponseDTOList(List<Nome> entities);

    @Mapping(target = "id", ignore = true)
    Nome toEntity(NomeRequestDTO dto);

    @Mapping(target = "id", ignore = true)
    void updateEntityFromDTO(
        NomeRequestDTO dto,
        @MappingTarget Nome entity
    );
}

```

Resumo final

Entity para ResponseDTO: usado para devolver dados pela API.

RequestDTO para Entity: usado para cadastrar dados recebidos.

@Mapping(ignore = true): usado para proteger campos que nao devem vir do usuario.

@MappingTarget: usado para atualizar uma entidade existente.

Service: aplica regra de negocio.

Mapper: faz conversao de dados.